description of methods

Pipeline для выделения аудиодорожек спикеров и расчёта формантных статистик из телефонных разговоров

Данный документ предназначен для понимания структуры пайплайна, используемого в данном репозитории. Здесь подробно описан алгоритм работы пайплайна, его этапы и используемые методы. На вход подаётся MP3-файл телефонного разговора с частотой дискретизации 8000 Гц. На выходе получаются отдельные аудиодорожки пользователя и оператора в более высоком разрешении (wav-формат), а также статистические параметры формант (F0, F1, F2, F3) для каждого спикера — среднее значение и дисперсия.

Общая схема пайплайна

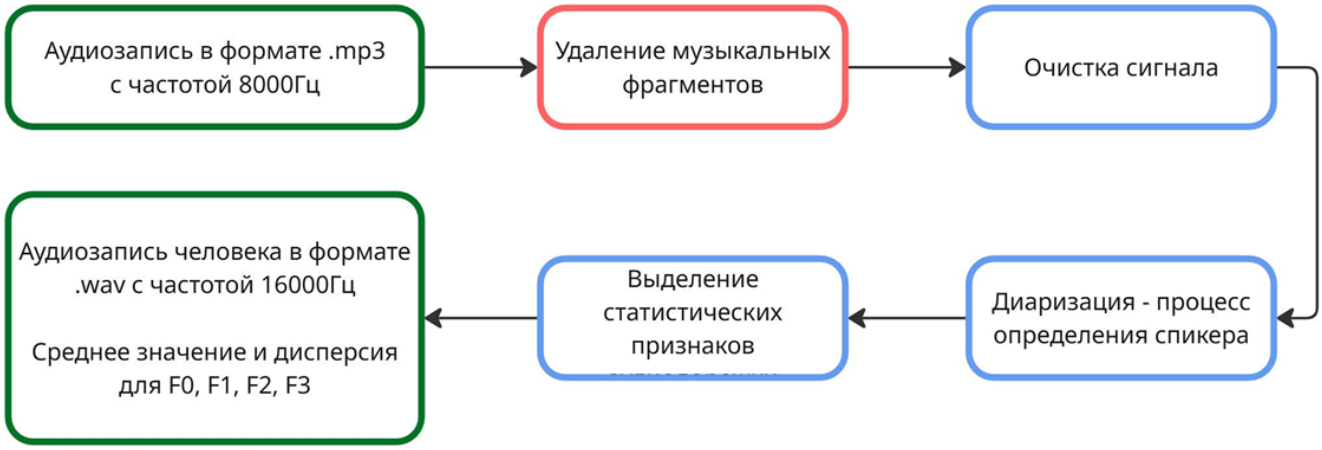

Рис.1 Общая схема пайплайна: от исходного MP3 до CSV-файла с формантными характеристиками спикеров.

Этап 1. Удаление музыкальных фрагментов

Первым этапом выполняется удаление участков с музыкой (обычно это фрагменты ожидания соединения с оператором). Для этого используется алгоритм динамической трансформации временной шкалы (DTW) — `librosa.sequence.dtw`. Чтобы повысить точность обнаружения именно музыкальных вставок, аудиосигнал предварительно преобразуется в **хром-признаки (chroma features)**. Хромы представляют собой 12-мерный вектор, соответствующий энергии в каждой из 12 полутоновых высот октавы (до, до# и т.д.). Такое представление инвариантно к тембру и громкости, но хорошо сохраняет гармоническую структуру, характерную для музыки. Благодаря этому алгоритм DTW

может надёжно сопоставлять музыкальные фрагменты, игнорируя различия в исполнении или качестве записи.

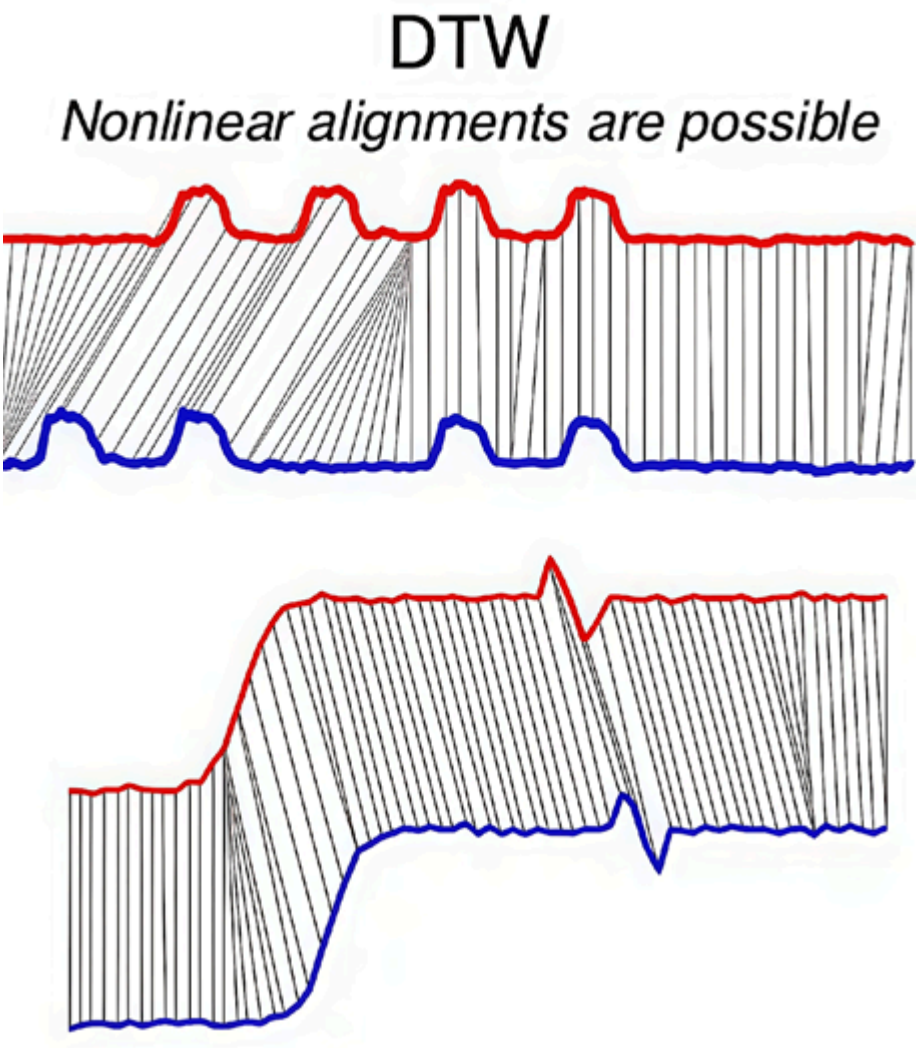

Рис.2 Принцип работы DTW.

Аудиофайл разбивается на трёхсекундные фреймы, и каждый сравнивается с заранее подготовленным эталонным фрагментом музыки. DTW возвращает меру схожести, по которой определяются стартовые точки музыкальных вставок.

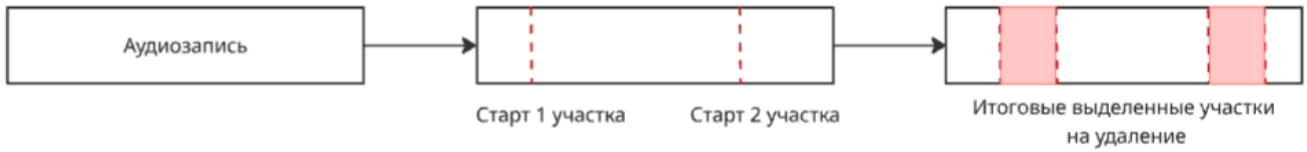

Рис.3 Определение границ музыкального фрагмента с помощью DTW.

Далее производится итеративное расширение границ: фреймы последовательно увеличиваются до тех пор, пока корреляция между исходным аудио и эталоном не упадёт ниже 75%. Как только значение корреляции становится меньше порога, алгоритм

останавливается, фиксируется конечная точка музыкального участка, и этот участок удаляется из дальнейшей обработки.

Этап 2. Очистка сигнала и предобработка

Для повышения качества распознавания речи и диаризации сигнал проходит несколько стадий обработки:

1. **Фильтр высоких частот (High-pass filter)** — удаляет частоты ниже 70 Гц, где обычно находятся инфранизкочастотные шумы (гул вентиляции, ветер, наводки электросети). Это позволяет избавиться от фонового шума, не затрагивая основной спектр речи (от 100–150 Гц).
2. **Мягкое усиление по среднеквадратичному значению (RMS-based gain)** — анализируется общая громкость всего сигнала. Если она ниже целевого уровня (–25 dBFS), добавляется равномерное усиление не более 8 дБ. Это выравнивает слишком тихие участки, например, когда спикер говорит вдалеке от микрофона.
3. **Покадровое усиление тихих участков (short-term adaptive gain)** — для каждого окна (с использованием окна Ханна для плавности) измеряется громкость. Если она ниже целевой (–28 dBFS), уровень слегка повышается (макс. 6 дБ). Громкие участки не меняются. Этот этап делает тихие фразы разборчивее, не искажая общий динамический диапазон.
4. **Мягкое сжатие (Soft limiter)** — применяется для предотвращения клиппинга и придания голосу большей насыщенности. Используется порог 0.98, а формула `tanh(y / threshold) * threshold` плавно ограничивает пики, не позволяя им превысить порог.
5. **Пиковая нормализация (Peak normalization)** — сигнал масштабируется так, чтобы максимальный по модулю отсчёт достигал значения 0.95 (чуть ниже абсолютного максимума 1.0). Это гарантирует максимальную громкость без риска клиппинга и стабилизирует уровень для последующей диаризации.

Все эти шаги выполняются с использованием библиотек `librosa` и `numpy`.

(Здесь будет картинка которая покажет каждый этап обработки!!!!)

Этап 3. Диаризация (определение спикеров)

Для разделения аудиопотока на голоса разных участников применяются две модели из библиотеки **pyannote**. Выбор модели зависит от сценария разговора:

- **pyannote 2.1** — для разговоров только с роботом (автоинформатор).
- **pyannote 3.1** — для разговоров с живым оператором.

Эмпирически доказано, что такая комбинация даёт наилучшие результаты: каждая модель оптимально работает в своём сценарии, а использование только одной из них не позволяет стабильно выделить все речевые сегменты.

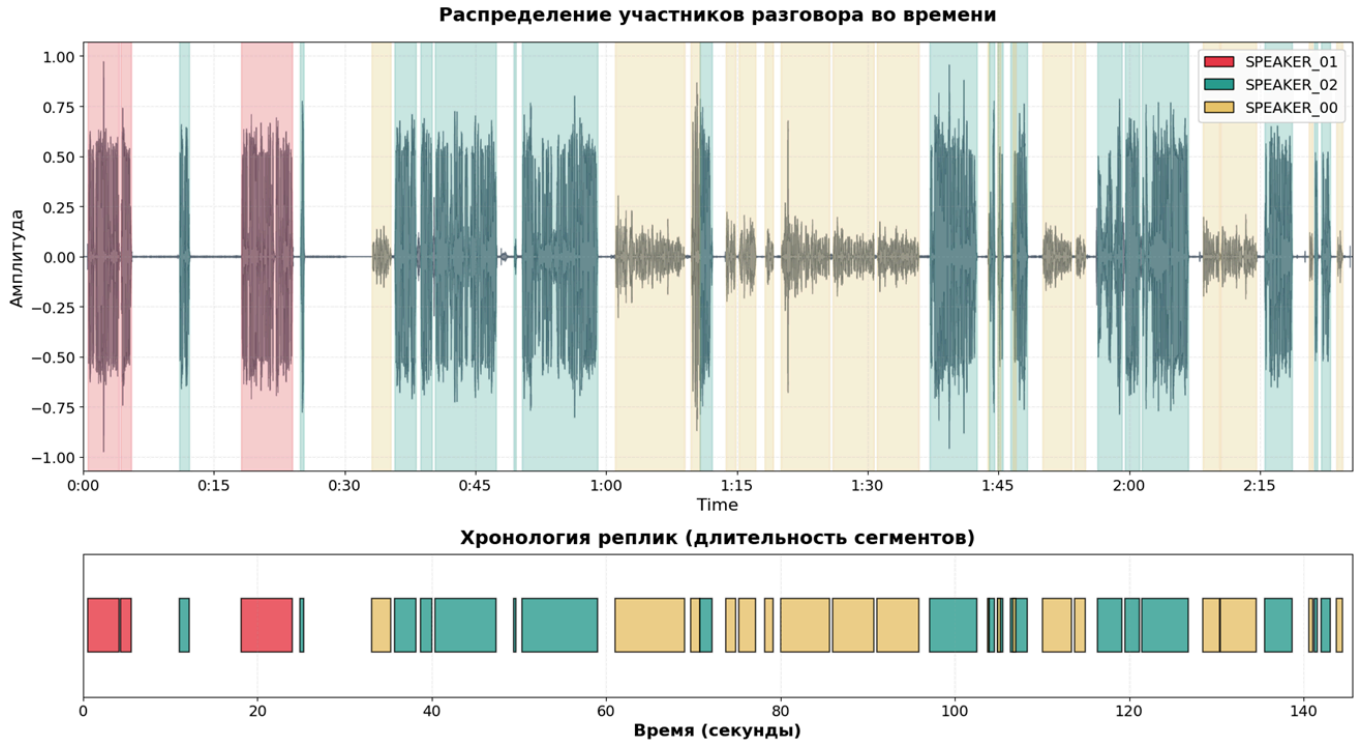

Рис.4 Пример диаризации: выделены сегменты робота, звонящего и оператора.

Модели pyannote возвращают размеченные сегменты, но не идентифицируют роли спикеров. Для маркировки используется модель **Sber GigaAM-v3**, выполняющая распознавание речи (audio-to-text) с сохранением орфографии и пунктуации. Она позволяет получить текст первых трёх фраз, произнесённых каждым спикером.

Далее эти фразы сравниваются с заготовленными шаблонами (типичные фразы робота, например: «Здравствуйте, это компания...», «Подтвердите, чтобы я связал вас...» и т.п.). Сравнение выполняется через **косинусное расстояние** между векторными представлениями фраз (эмбеддингами). Для получения этих векторных представлений используется библиотека **sentence_transformers**, которая предоставляет удобный интерфейс для работы с предобученными моделями. Sber-модель преобразует каждый текст в вектор фиксированной размерности, сохраняя семантическую близость. Формула косинусного расстояния:

$$\text{cosine_distance}(A, B) = 1 - \frac{A \cdot B}{\|A\| \|B\|}$$

где A и B — векторы сравниваемых фраз. Чем меньше расстояние, тем более похожи фразы. Порог срабатывания установлен на **0.85**, что гарантирует высокую уверенность в совпадении с шаблоном.

После идентификации робота остальные спикеры маркируются по принципу «кто первый, тот пользователь». Далее из общего аудио выделяются две отдельные дорожки: пользователя и оператора (если оператор присутствовал в разговоре).

Этап 4. Извлечение формантных характеристик

Для анализа голосовых параметров используется Python-библиотека **Parselmouth** — интерфейс к программе **Praat**. С её помощью из каждой выделенной аудиодорожки извлекаются значения формант F0, F1, F2, F3 (частоты основного тона и первых трёх формант). Для этого применяются встроенные алгоритмы Praat (анализ на основе LPC).

Для каждой форманты получается массив значений во временной области. Затем рассчитываются статистики:

- **Среднее значение** — характеризует типичную частоту форманты у спикера.
- **Дисперсия** — отражает вариативность форманты в речи.

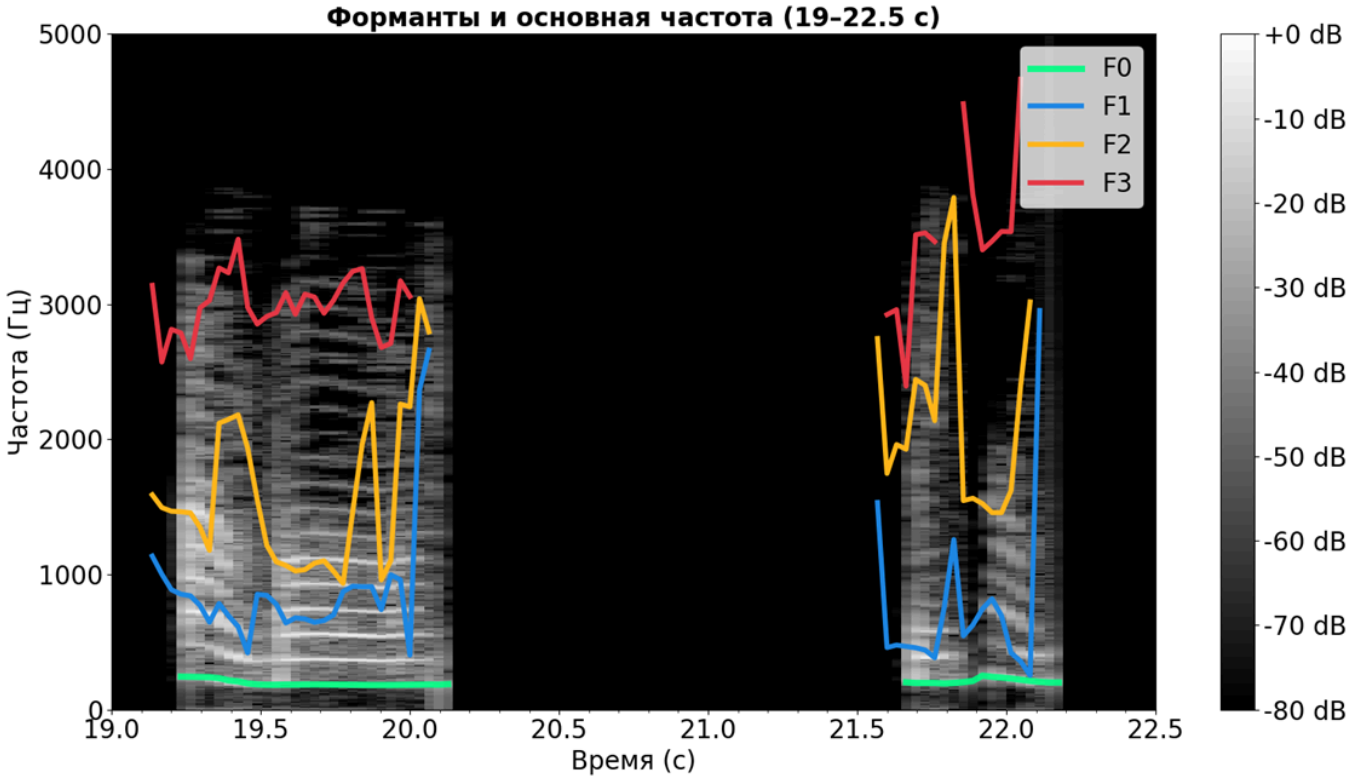

Рис.5 Разделение на форманты полученного сигнала пользователя на небольшом промежутке 3.5 секунды.

Этап 5. Сохранение результатов

Результатом обработки является CSV-файл со следующими полями:

call_id	role	f0_mean	f0_variance	fn_mean	fn_variance
...	...	...	...	...	...

n = 1, 2, 3

- **call_id** — уникальный идентификатор разговора (8-значный хеш от имени исходного аудиофайла).
- **role** — роль спикера: `user` или `operator`.

Перед заполнением таблицы для каждого исходного аудиофайла создаётся 8-значный хеш. Соответствие между именем файла и хешем сохраняется в отдельном JSON-файле в формате словаря:

```
{
    "audio_file_1.mp3": "a1b2c3d4",
    "audio_file_2.mp3": "e5f6g7h8",
    ...
}
```

Заключение

Представленный пайплайн реализует полный цикл обработки телефонного разговора: от очистки и разделения спикеров до вычисления формантных статистик. Несмотря на то, что решение не является оптимальным по производительности, оно успешно решает поставленную задачу — позволяет надёжно идентифицировать пользователя и оператора, а затем извлечь их голосовые характеристики для дальнейшего анализа.

Все описанные методы и подходы можно адаптировать под конкретные условия, изменяя пороговые значения, модели диаризации или шаблоны фраз. Код сопровождается комментариями и готов к использованию в исследовательских и прикладных проектах.

(Если в процессе чтения у вас возникнут вопросы или пожелания по улучшению, создавайте issue в репозитории.)